Intelligenza Artificiale

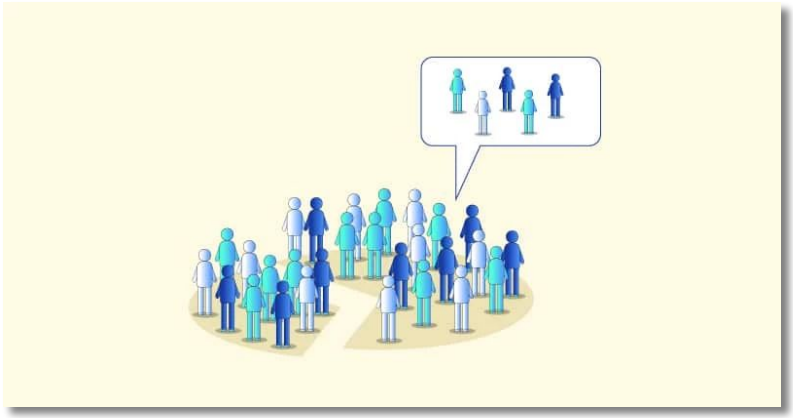
“Bayesian networks II”

Nicola Bellotto

A.A. 2025/2026

Lecture outline

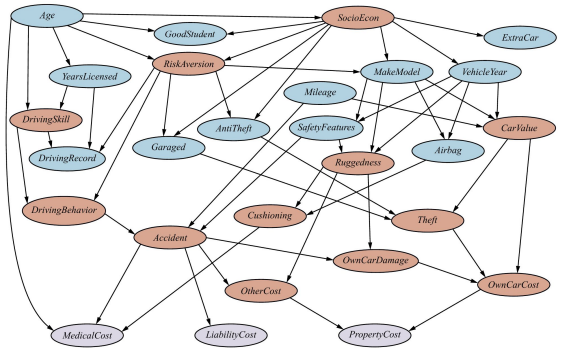
- Approximate inference
- Probabilistic sampling
- Prior sampling
- Rejection sampling
- Likelihood Weighting
- Gibbs sampling

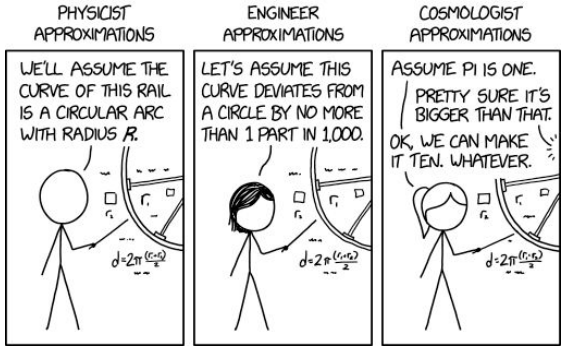


Slides partly based on Russell & Norvig instructors material http://aima.cs.berkeley.edu/instructors.html

Exact vs Approximate Inference

- The methods seen so far (i.e. inference by enumeration) allow for exact inference
- They become problematic though in case of large probability models, mainly because of computational complexity
- **Approximate inference** methods are therefore a viable alternative to give reasonable answers in case of large models

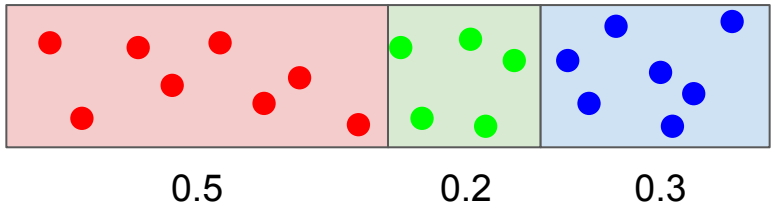


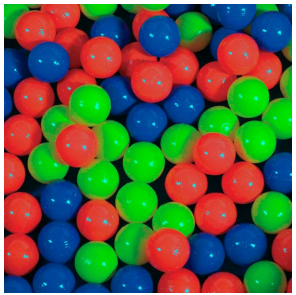


Probabilistic Sampling

- The main idea is to draw N samples from the probability distributions of the network, and from those derive the distribution of the query $\mathbf{P}(X \mid evidence)$
- Example: sampling 20 times from a probability distribution

$$\mathbf{P}(C) = \langle P(red), P(green), P(blue) \rangle = \langle 0.5, 0.2, 0.3 \rangle$$

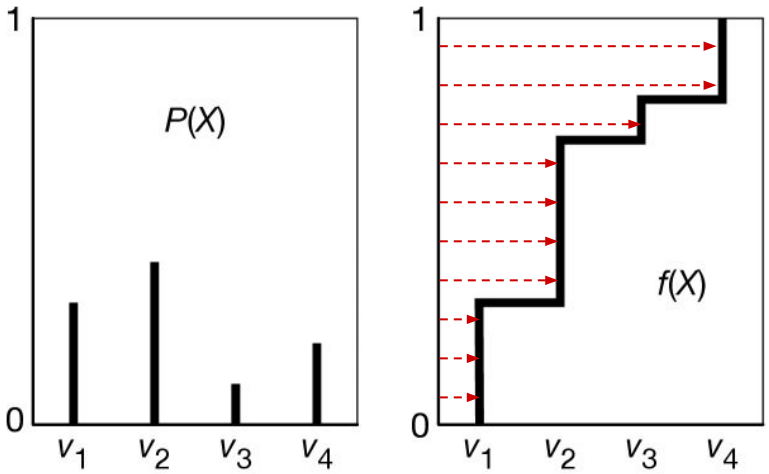




⇒ approximate distribution $\hat{\mathbf{P}}(C) = \langle \frac{8}{20}, \frac{5}{20}, \frac{7}{20} \rangle = \langle 0.4, 0.25, 0.35 \rangle$

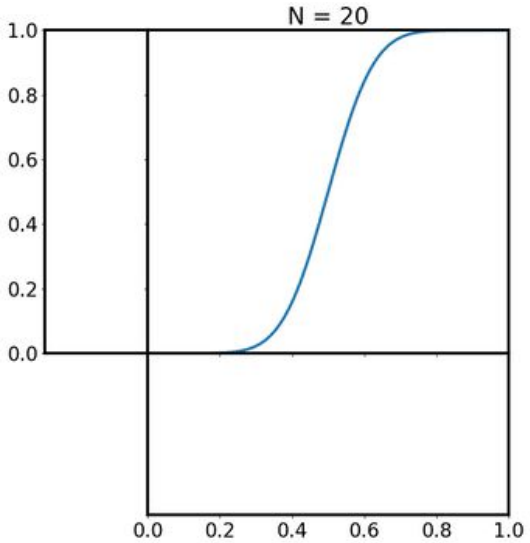
Probabilistic Sampling

Choose a uniformly random value from the cumulative probability distribution and take the inverse



(from *Poole & Mackworth "Artificial Intelligence"*)

E.g. for a Gaussian variable



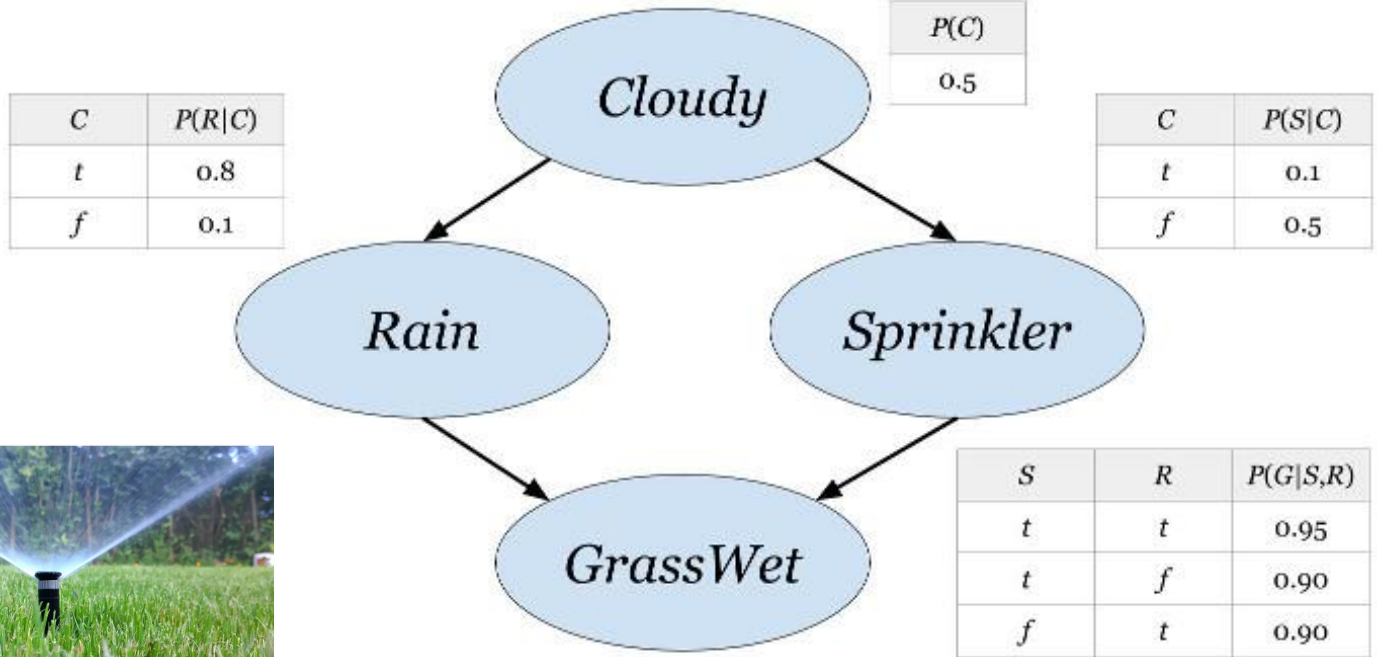
(from *Wikipedia*)

Monte Carlo algorithms

- As the number of sample increases, the approximation converges to the true probability distribution
- Methods based on randomized sampling are called **Monte Carlo** algorithms
- Next, we'll consider two classes:
  - **Direct sampling** methods (e.g. **prior sampling**, **rejection sampling**, **likelihood weighting**)
  - **Markov Chain Monte Carlo** (**MCMC**) methods (e.g. **Gibbs sampling**)

- We'll use the classic "sprinkler" example to explain the algorithms

Sampling from Bayesian Networks



$P(C)$
0.5

C	$P(R\|C)$
t	0.8
f	0.1

C	$P(S\|C)$
t	0.1
f	0.5

S	R	$P(G\|S,R)$
t	t	0.95
t	f	0.90
f	t	0.90
f	f	0.10

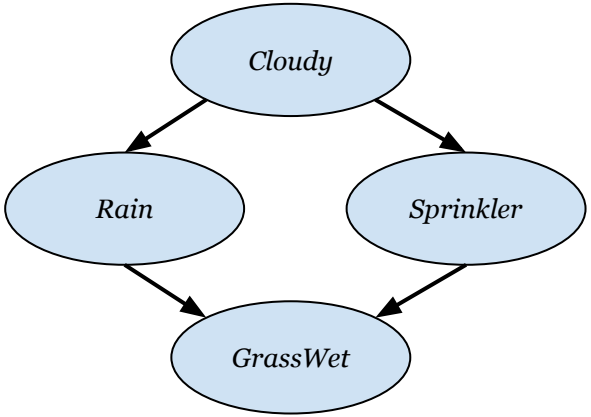
Prior Sampling

- The simplest sampling generates events from a network without evidence

- Assuming the following topological order:

  $[Cloudy, Sprinkler, Rain, GrassWet]$

  1. generate sample from $\mathbf{P}(Cloudy)$ → e.g. suppose to obtain ***true***
  2. sample from $\mathbf{P}(Sprinkler \mid Cloudy = true)$ → ***false***
  3. sample from $\mathbf{P}(Rain \mid Cloudy = true)$ → ***true***
  4. sample from $\mathbf{P}(GrassWet \mid Sprinker = false, Rain = true)$ → ***true***



  - The output of this initial sample generation of $\mathbf{P}(Cloudy, Sprinkler, Rain, GrassWet)$ will be [***true***, ***false***, ***true***, ***true***]

Prior Sampling

- If we repeat the sampling many times (large N), we expect something like

$[true, false, true, true]$

$[true, true, false, true]$

...

$[false, false, true, false]$

$$\approx \mathbf{P}(Cloudy, Sprinkler, Rain, GrassWet)$$

Follows from the BN in slide 7:
$P(c, \neg s, r, g) = P(c)\, P(\neg s \mid c)\, \mathrm{P}(r \mid c)\, \mathrm{P}(g \mid \neg s, r)$

- For example, since $P(c, \neg s, r, g) = 0.5 \times 0.9 \times 0.8 \times 0.9 = 0.324$ we expect that 32.4% of the N samples will be $[true, false, true, true]$

Prior Sampling

We can use simple random generators – example in the lab

function PRIOR-SAMPLE(bn) **returns** an event sampled from the prior specified by bn
  inputs: bn, a Bayesian network specifying joint distribution $\mathbf{P}(X_1, \ldots, X_n)$

  $\mathbf{x} \leftarrow$ an event with n elements
  for each variable X_i **in** $X_1, \ldots, X_n$ **do**
    $\mathbf{x}[i] \leftarrow$ a random sample from $\mathbf{P}(X_i \mid parents(X_i))$
  return x

- With N samples from the above function, the probability of an event is

$$P(x_1, x_2, \ldots, x_n) \approx \frac{N_{PS}(x_1, x_2, \ldots, x_n)}{N}$$

- E.g. if we generate 1000 samples and in 511 of them $Rain = true$, then

$$\hat{P}(r) = 511/1000 = 51.1\%$$

(the hat ^ means “approximated”)

P(Cloudy = true, Rain = true) = ?

0

Cloudy	*Sprinkler*	*Rain*	*GrassWet*
TRUE	FALSE	TRUE	TRUE
TRUE	FALSE	FALSE	FALSE
FALSE	TRUE	TRUE	FALSE
TRUE	FALSE	TRUE	TRUE
FALSE	FALSE	FALSE	FALSE
TRUE	TRUE	TRUE	TRUE
TRUE	TRUE	FALSE	FALSE
TRUE	TRUE	TRUE	FALSE
FALSE	FALSE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE

0.40 — 0

0.50 — 0

0.67 — 0

0.83 — 0

None of the above — 0

P(Cloudy = true, Rai = true) = 4/10 = 0.4

Cloudy	*Sprinkler*	*Rain*	*GrassWet*
TRUE	FALSE	TRUE	TRUE
TRUE	FALSE	FALSE	FALSE
FALSE	TRUE	TRUE	FALSE
TRUE	FALSE	TRUE	TRUE
FALSE	FALSE	FALSE	FALSE
TRUE	TRUE	TRUE	TRUE
TRUE	TRUE	FALSE	FALSE
TRUE	TRUE	TRUE	FALSE
FALSE	FALSE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE

Rejection Sampling

- Let's assume we have some evidence $\mathbf{e}$ and want to compute $\mathbf{P}(X \mid \mathbf{e})$

$$\hat{\mathbf{P}}(X|\mathbf{e}) = \frac{\hat{\mathbf{P}}(X,\mathbf{e})}{\hat{P}(\mathbf{e})} = \alpha\hat{\mathbf{P}}(X,\mathbf{e}) = \alpha\frac{\mathbf{N}_{PS}(X,\mathbf{e})}{N} = \beta\mathbf{N}_{PS}(X,\mathbf{e}) \approx \mathbf{P}(X|\mathbf{e})$$

  where α and β are just normalisation factors

- The idea therefore is to generate samples like before, but counting only those which are consistent with the evidence, and rejecting the others

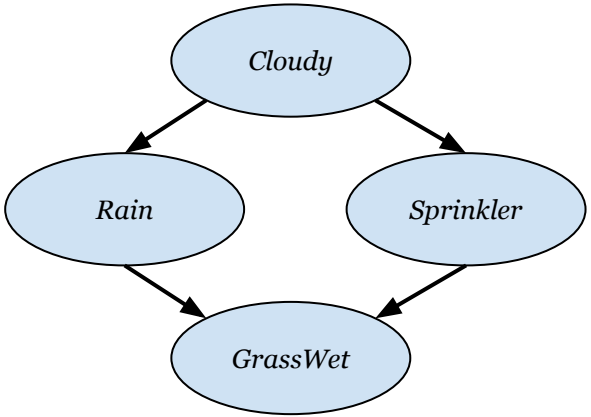
Rejection Sampling: algorithm

function REJECTION-SAMPLING(X, **e**, bn, N) **returns** an estimate of $\mathbf{P}(X \mid \mathbf{e})$
  inputs: X, the query variable
    e, observed values for variables **E**
    bn, a Bayesian network
    N, the total number of samples to be generated
  local variables: **C**, a vector of counts for each value of X, initially zero

  for $k = 1$ **to** N **do**
    x $\leftarrow$ PRIOR-SAMPLE(bn)
    if x is consistent with **e then**
      C[j] $\leftarrow$ **C**[j]+1 where x_j is the value of X in **x**
  return NORMALIZE(**C**)

- Note that the algorithm calls the same **Prior-Sample** function as before, but now counts only those cases which are consistent with the given evidence

Rejection Sampling

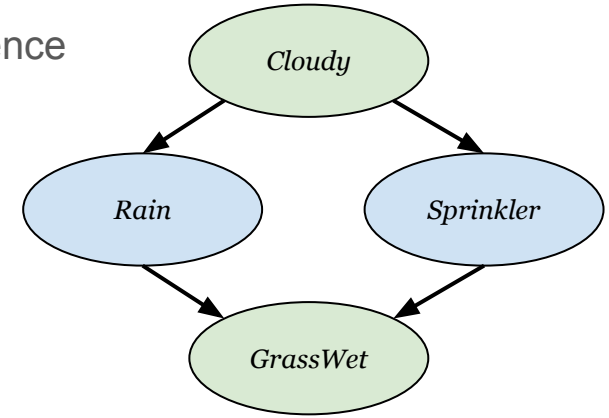


- E.g. estimate $\mathbf{P}(Rain \mid Sprinkler = true)$
  - generate 100 samples with prior-sampling
  - let's assume that 73 have $Sprinkler = false$, and are therefore **rejected**
  - of the remaining 27 with $Sprinkler = true$, 8 have $Rain = true$ and 19 have $Rain = false$
  - ⇒ the (approximate) conditional distribution is

$$\hat{\mathbf{P}}(Rain|Sprinkler = true) = \beta \langle 8, 19 \rangle = \langle 0.296, 0.704 \rangle$$

Likelihood Weighting

- The problem of rejection sampling is that it wastes a lot of samples!
- Key idea: generate only samples that are consistent with the evidence $\mathbf{e}$ by
  - fixing the values of the evidence nodes $\mathbf{E}$ in the network
  - sampling the non-evidence variables
  - reward samples that are more consistent with the evidence



- E.g. if the query is $\mathbf{P}(Rain \mid c, g)$, then
  - Fix the values $Cloudy = true$ and $GrassWet = true$
  - Sample only $Rain$ and $Sprinkler$

Likelihood Weighting: algorithm

function LIKELIHOOD-WEIGHTING(X, **e**, bn, N) **returns** an estimate of $\mathbf{P}(X \mid \mathbf{e})$
  inputs: X, the query variable
    e, observed values for variables **E**
    bn, a Bayesian network specifying joint distribution $\mathbf{P}(X_1, \ldots, X_n)$
    N, the total number of samples to be generated
  local variables: **W**, a vector of weighted counts for each value of X, initially zero

  for $j = 1$ **to** N **do**
    x, $w \leftarrow$ WEIGHTED-SAMPLE(bn, **e**)
    $\mathbf{W}[j] \leftarrow \mathbf{W}[j] + w$ where x_j is the value of X in **x**
  return NORMALIZE(**W**)

function WEIGHTED-SAMPLE(bn, **e**) **returns** an event and a weight

  $w \leftarrow 1$; **x** $\leftarrow$ an event with n elements, with values fixed from **e**
  for $i = 1$ **to** n **do**
    if X_i is an evidence variable with value x_{ij} in **e**
      then $w \leftarrow w \times P(X_i = x_{ij} \mid parents(X_i))$
      else **x**$[i] \leftarrow$ a random sample from $\mathbf{P}(X_i \mid parents(X_i))$
  return **x**, w

Likelihood Weighting: example

Query $\mathbf{P}(Rain \mid c, g)$, assuming topological order $[Cloudy, Sprinkler, \underline{Rain}, GrassWet]$

- Initial weight $w \leftarrow 1$
- $Cloudy$ is evidence, therefore $w \leftarrow w \times P(c) = 1 \times 0.5 = 0.5$
- $Sprinkler$ is not evidence, therefore sample from $\mathbf{P}(Sprinkler \mid c)$ – let's assume ***false***
- $Rain$ is not evidence, therefore sample from $\mathbf{P}(Rain \mid c)$ – let's assume ***true***
- $GrassWet$ is evidence, therefore $w \leftarrow w \times P(g \mid \neg \mathbf{s}, \mathbf{r}) = 0.5 \times 0.9 = 0.45$
- Return event $[true, \mathbf{false}, \mathbf{true}, true]$ with weight $w = \mathbf{0.45}$

The sample is therefore weighted by the likelihood of the evidence!

Likelihood Weighting: example

Let's repeat this $N = 10$ times

Cloudy (fixed)	*Sprinkler*	*Rain*	*GrassWet* (fixed)	w
true	***false***	***true***	*true*	0.45
true	***true***	***true***	*true*	0.475
true	***false***	***false***	*true*	0.05
true	***false***	***true***	*true*	0.45
true	***false***	***true***	*true*	0.45
true	***false***	***true***	*true*	0.45
true	***true***	***true***	*true*	0.475
true	***false***	***true***	*true*	0.45
true	***false***	***false***	*true*	0.05
true	***false***	***true***	*true*	0.45

Likelihood Weighting: example

EXCERCISE:
verify that these results are similar to the exact probabilities

- Sum and normalize all the weights for which *Rain* = *true*

$$P(r|c,g) = \frac{\sum_r w}{\sum w}$$
$$= \frac{0.45 + 0.475 + 0.45 + 0.45 + 0.45 + 0.475 + 0.45 + 0.45}{0.45 + 0.475 + 0.05 + 0.45 + 0.45 + 0.45 + 0.475 + 0.45 + 0.05 + 0.45}$$
$$= 0.9733$$

- Sum and normalize all the weights for which *Rain* = *false*

$$P(\neg r|c,g) = \frac{\sum_{\neg r} w}{\sum w}$$
$$= \frac{0.05 + 0.05}{0.45 + 0.475 + 0.05 + 0.45 + 0.45 + 0.45 + 0.475 + 0.45 + 0.05 + 0.45}$$
$$= 0.0267$$

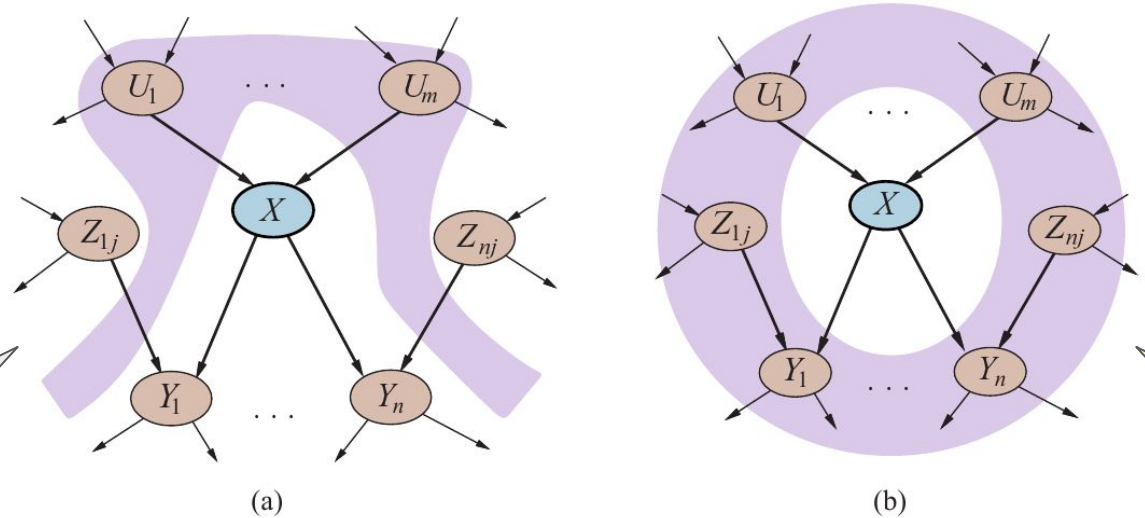
Obviously equal to $1 - P(r \mid c, g)$

Markov Chain Monte Carlo (MCMC) methods

- Likelihood Weighting is generally more efficient than Rejection Sampling because it uses only useful samples
- Its performance decreases though as the number of variable grows because the weights become smaller and smaller…

- **IDEA**: Instead of sampling every time from scratch, **Markov Chain Monte Carlo** (MCMC) methods generate each samples by randomly changing the previous one – that's why it's called a "chain"…

Markov Blanket



Non-descendant property

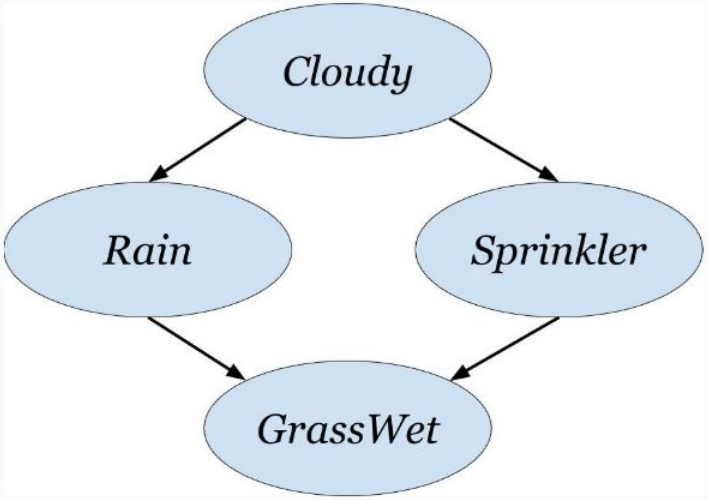
Markov blanket property

Figure 13.4 (a) A node X is conditionally independent of its non-descendants (e.g., the Z_{ij}s) given its parents (the U_is shown in the gray area). (b) A node X is conditionally independent of all other nodes in the network given its Markov blanket (the gray area).

i.e. parents, children, and other parents of its children

Markov Blanket of Sprinkler

0



Cloudy

0

Rain

0

GrassWet

0

None of the above

0

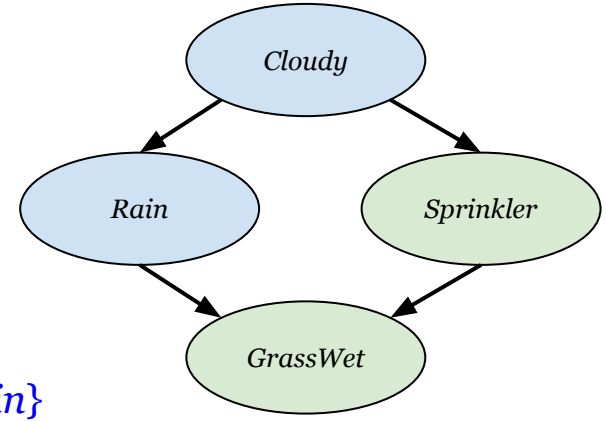
Gibbs Sampling: algorithm

- **Gibbs Sampling** is a particular MCMC method that works as follows:
  - Starting from an arbitrary **state** event, and given some evidence $\mathbf{e}$, generate the next state by sampling a value from one of the non-evidence variable Z_i **given its Markov Blanket** $mb(Z_i)$
  - Wander around the state space sampling one non-evidence variable at a time, while keeping the others fixed

function GIBBS-ASK(X, **e**, bn, N) **returns** an estimate of $\mathbf{P}(X \mid \mathbf{e})$
  local variables: **C**, a vector of counts for each value of X, initially zero
    Z, the nonevidence variables in bn
    x, the current state of the network, initialized from **e**

  initialize **x** with random values for the variables in **Z**
  for $k = 1$ **to** N **do**
    choose any variable Z_i from **Z** according to any distribution $\rho(i)$
    set the value of Z_i in **x** by sampling from $\mathbf{P}(Z_i \mid mb(Z_i))$
    $\mathbf{C}[j] \leftarrow \mathbf{C}[j] + 1$ where x_j is the value of X in **x**
  return NORMALIZE(**C**)

Gibbs Sampling: example



Query $\mathbf{P}(Rain \mid s, g)$

- Let's assume initial state $[c, s, \neg r, g]$
  1. Sample $Cloudy$ given its Markov Blanket $\{Sprinkler, Rain\}$
     $\mathbf{P}(Cloudy \mid s, \neg r) \rightarrow$ let's assume we get ***false***, then the new state is $[\neg \mathbf{c}, s, \neg r, g]$
  2. Sample $Rain$ given it's Markov Blanket $\{Cloudy, Sprinkler, Grass\}$
     $\mathbf{P}(Rain \mid \neg c, s, g) \rightarrow$ let's assume we get ***true***, then the new state is $[\neg c, s, \mathbf{r}, g]$
  3. (continue by randomly choosing $Cloudy$ or $Rain$ …)
- E.g., if we run this 80 times, and 20 times $Rain = true$, 60 times $Rain = false$

$$\hat{\mathbf{P}}(R|s, g) = \alpha\langle 20, 60\rangle = \langle 0.25, 0.75\rangle = \langle P(r|s, g), P(\neg r|s, g)\rangle$$

Conclusions

- Suggested reading
  - Chapter 13, sec. 13.4 of Russell & Norvig
- Next lecture

  Probabilistic reasoning over time
- Questions?